

APIx — Technology Stack

Team CodeCrew | Smart India Hackathon 2026 | PS ID: SIH26056

Every entry below was checked directly against `requirements.txt`, the `Dockerfile` / `docker-compose.yml`, and the actual import/execution paths in the code — not just against what's *mentioned* in comments or docstrings. Where a technology is present in the codebase but doesn't actually execute in the current build, that's called out explicitly rather than glossed over, especially for the scraping layer (see Section 3), which turned out to be more nuanced than a simple "Playwright + Scrapy" summary.

1. Language & Runtime

Component	Choice	Notes
Language	Python 3.11 (Docker image: <code>python:3.11-slim</code>)	Single-language stack across scraping, cleaning, index math, API, and dashboard
Package management	<code>pip + requirements.txt</code>	No <code>pyproject.toml</code> /Poetry; version floors only (<code>>=</code>), one exact pin (<code>streamlit==1.51.0</code>)

2. Core Application Dependencies (from `requirements.txt` — actually installed)

Package	Version constraint	Used for
<code>fastapi</code>	<code>≥0.110.0</code>	REST API framework (12 endpoints)
<code>uvicorn</code>	<code>≥0.28.0</code>	ASGI server running the FastAPI app
<code>starlette</code>	<code>≥0.36.0</code>	FastAPI's underlying ASGI toolkit (explicit pin, not just a transitive dep)
<code>pydantic</code>	<code>≥2.5.0</code>	Request/response validation & schema models
<code>streamlit</code>	<code>==1.51.0</code>	Dashboard framework (9 tabs)
<code>plotly</code>	<code>≥5.19.0</code>	Interactive charts inside the dashboard
<code>pandas</code>	<code>≥2.0.0</code>	DataFrame operations across cleaning, index computation, and dashboard
<code>numpy</code>	<code>≥1.24.0</code>	Numerical computation (RMSE/MAPE, array math)

<code>scipy</code>	<code>≥1.11.0</code>	<code>scipy.stats.pearsonr</code> for backtest correlation
<code>requests</code>	<code>≥2.31.0</code>	HTTP client for the (attempted) live airline scraping path and <code>robots.txt</code> fetches
<code>pytest</code>	<code>≥8.0.0</code>	Test runner (27 tests, 7 modules)
<code>psutil</code>	<code>≥5.9.0</code>	Process management for <code>start_services.py</code> / <code>stop_services.py</code> launcher scripts

Standard library only, no extra dependency: `sqlite3` (database), `urllib.robotparser` + `urllib.parse` (`robots.txt` compliance — the actual mechanism, not a third-party package), `logging`, `time` / `random` (rate-limit jitter).

3. Data Acquisition Layer — what's actually live vs. scaffolded

This is worth being precise about, because the codebase presents four different scraper implementations with genuinely different levels of "live-ness," and it matters for accurately describing the system.

Scraper class	File	What it actually does when called
<code>AirlinePortalScraper</code> (5 airlines)	<code>airline_scrapers.py</code>	Genuinely attempts one real HTTP GET via <code>requests</code> to <code>{base_url}/api/search</code> with real query params, a 4-second timeout, and rotated headers, gated by a real <code>robots.txt</code> check. Falls back to the synthetic engine on any non-200 response, 403/429, or exception. In practice, this will almost always fall back against real airline sites, since <code>/api/search</code> is a guessed path, not each airline's actual (typically authenticated GDS/NDC) API contract — but the attempt itself, the <code>robots.txt</code> gate, and the fallback logic are all real, executing code.
<code>OTAPortalScraper</code> (5 OTAs)	<code>ota_scrapers.py</code>	Does not attempt any live HTTP request at all. Goes straight to the synthetic engine every time, applying a convenience-fee markup on top. The only "real" behavior here is the rate-limit delay (<code>polite_wait</code>) before generating the synthetic quote.
		Browser automation is fully commented out. The real Playwright

PlaywrightAirlineScraper	playwright_scraper.py	code (sync_playwright() , browser.new_page() , network interception) exists only as a comment describing the intended production design. The live code path does import playwright purely to check if the package is installed (it logs either way), then unconditionally returns synthetic data regardless of that check's outcome. playwright is not in requirements.txt .
AirfareSpider (Scrapy)	spiders/airfare_spider.py	A real, importable Scrapy spider with genuinely correct ethical settings configured (ROBOTSTXT_OBEY=True , AUTOTHROTTLE_ENABLED , DOWNLOAD_DELAY=1.5s), but its start_requests() targets https://httpbin.org/get?... — a public request-echoing test service, not any real airline/OTA site. scrapy is not in requirements.txt , so this spider would need a separate install to even run.
DynamicPricingEngine (synthetic engine)	synthetic_engine.py	The actual, dominant data source in every current execution path — calibrated fare curves by distance, advance-purchase window, and carrier, used directly by 3 of the 4 scraper classes and as the fallback for the 4th.

Honest one-line summary: the *architecture* for ethical, multi-source live scraping (robots.txt compliance, rate limiting, UA rotation, Playwright/Scrapy scaffolding) is real and well-designed, but in the **current build**, effectively 100% of the data flowing through the pipeline at runtime comes from the calibrated synthetic engine — either because a scraper never attempts a live call at all (OTAs, Playwright), or because the one scraper that does attempt one (airlines) is very likely to fail against a real site's actual API and fall back gracefully. This is worth stating plainly rather than letting "Playwright + Scrapy" imply more live-scraping activity than the code currently performs.

Ethical scraping guardrails that are fully real and active regardless of the above:

EthicalScrapingGuard in base_scraper.py — robots.txt parsing via urllib.robotparser , per-domain rate limiting with jitter, a curated rotating pool of 4 realistic user-agents, and polite exponential back-off — this layer runs for every scraper class, live or fallback.

4. Data Processing & Index Computation

--	--	--

Layer	Technology	Detail
Cleaning	<code>pandas</code>	Deduplication (direct-airline priority), stratified IQR outlier filtering (per route × window), sold-out imputation
Fare decomposition	<code>pandas</code> + hardcoded config constants	GST (5%/12%), UDF (₹300–480 across 18 airports), PSF (₹91–110), OTA fee (₹0–399)
Index formulas	Custom Python (<code>apix/index/formulas.py</code>)	<code>two_tier_jevons_laspeyres_index</code> (the production/"champion" formula) + <code>jevons_index</code> (genuinely independent) + <code>laspeyres_index</code> / <code>paasche_index</code> / <code>fisher_ideal_index</code> (aliased/derived from the same aggregation function — see the Test Stack document's nuance note)
Route weighting	<code>pandas</code> reading <code>data/dgca_route_weights.csv</code>	30 corridors, DGCA passenger-traffic-derived weights
Regulatory watchdog	Custom Python (<code>apix/index/watchdog.py</code>)	4-tier surge/cap classification, config-driven thresholds
eSankhyiki/CPI augmentation	Custom Python (<code>apix/index/esankhyiki.py</code>)	COICOP 2018-aligned weight structure, 2012=100 rebasing
Backtest validation	<code>scipy.stats.pearsonr</code> + NumPy	Live-computed r/MAPE/RMSE against a hardcoded 3-month DGCA benchmark

5. Persistence

Component	Technology	Detail
Database	SQLite (<code>sqlite3</code> , <code>stdlib</code>)	Single file (<code>data/apix.db</code>); tables for raw quotes, cleaned quotes, daily index (5 formula columns, all <code>NOT NULL</code>), DGCA benchmarks, route weights
Schema definitions	<code>apix/database/models.py</code>	Plain Python dataclass-style schema, no ORM (no SQLAlchemy)
Data access	<code>apix/database/db.py</code>	Hand-written SQL via <code>sqlite3</code> , not a query builder

6. API Layer

Component	Technology	Detail
Framework	FastAPI	12 endpoints across System, Airfare Price Index, Metadata & Weights, Analytics, Econometric Validation, eSankhyiki Integration,

		Scraping Engine, and Watchdog tags
Server	Uvicorn	ASGI server, run via <code>uvicorn apix.api.server:app --host 0.0.0.0 --port 8000</code>
Validation	Pydantic	Request/response models
Docs	Auto-generated OpenAPI/Swagger (<code>/docs</code>) and ReDoc	Free from FastAPI, no separate documentation tooling
CORS	FastAPI's built-in <code>CORSMiddleware</code>	Open cross-origin access; no authentication layer currently implemented

7. Dashboard / Frontend

Component	Technology	Detail
Framework	Streamlit	9 tabs: Daily Trajectory, MoSPI vs APIx, Corridors & Map, Dynamic Pricing, DGCA Watchdog, Statutory Taxes, DGCA Backtest, Ethical Scraper, MoSPI & RBI Export
Charts	Plotly	All interactive visualizations
Styling	Custom CSS injected via <code>st.markdown(..., unsafe_allow_html=True)</code>	No separate frontend framework (no React/Vue)
Caching	<code>st.cache_data(ttl=60)</code>	Index data reloaded from SQLite at most once per minute
Theming	<code>.streamlit/config.toml</code>	Custom dark theme (<code>primaryColor #38BDF8</code> , dark navy background <code>#080C15</code>), CORS/XSRF explicitly disabled at the Streamlit-server level (separate from FastAPI's own CORS setting) for local dev convenience

8. Testing

Component	Technology	Detail
Framework	pytest ($\geq 8.0.0$)	27 test functions across 7 modules
API testing	FastAPI TestClient	In-process HTTP testing, no live server needed
Coverage tooling	None installed	No <code>pytest-cov</code> / <code>coverage.py</code> in <code>requirements.txt</code>

CI	None configured	No <code>.github/workflows</code> , no <code>conftest.py</code> , no <code>pytest.ini</code> — tests are run manually via <code>python -m pytest tests/ -v</code>
----	-----------------	---

(Full per-test breakdown lives in the separate [Test Stack](#) document.)

9. Deployment & Infrastructure

Component	Technology	Detail
Containerization	Docker (<code>python:3.11-slim</code> base image)	Single Dockerfile builds both the API and dashboard images (different <code>command</code> per service)
Orchestration	docker-compose (v3.8)	3 services: <code>api</code> (port 8000), <code>dashboard</code> (port 8501), <code>nginx</code> (port 80) — each with a <code>healthcheck</code> block
Reverse proxy	Nginx (<code>nginx:alpine</code>)	Routes <code>/api/</code> , <code>/docs</code> , <code>/openapi.json</code> to the FastAPI upstream and everything else to the Streamlit upstream, unifying both services behind port 80
Scheduling (containerized)	<code>docker-entrypoint.sh</code> + <code>scripts/start_services.py</code> / <code>stop_services.py</code>	Process launch/teardown, using <code>psutil</code>
Scheduling (bare-metal)	<code>systemd timer</code> (<code>scripts/setup_timer.sh</code>)	Triggers scrape runs at fixed intraday epochs
Local (non-Docker) setup	<code>run.sh</code> (Linux/Mac) and <code>run.bat</code> (Windows), project root	Both create a Python venv (<code>.venv</code>), install <code>requirements.txt</code> , run <code>scripts/init_db.py</code> to check/seed the database, then launch <code>scripts/start_services.py</code> — a genuine one-command local setup path independent of Docker, not just a Docker-only project
Cloud deployment	AWS	Live demo instance used during development (per project README)
OS packages (in	<code>curl</code> , <code>gcc</code> , <code>libffi-dev</code>	<code>curl</code> for the compose healthchecks; <code>gcc</code> / <code>libffi-dev</code>

Docker image)		as native build dependencies for Python packages
---------------	--	--

10. External Data Sources (not code dependencies, but inputs)

Source	Role
DGCA (Directorate General of Civil Aviation)	Route/corridor passenger-traffic weights, monthly benchmark fare data for backtesting
India Aviation Traffic dataset (github.com/Vonter/india-aviation-traffic , open-source)	Cross-verification source for DGCA traffic figures
MoSPI / eSankhyiki	Target schema for the CPI augmentation export (not an inbound dependency — APIx produces output <i>for</i> this system)
CBIC	GST slab rates (5% economy / 12% business) referenced in decomposition logic

11. What's Deliberately Not in the Stack

- **No ORM** (SQLAlchemy, etc.) — raw SQL via `sqlite3`.
- **No message queue** (Celery, RabbitMQ, Kafka) — scheduling is handled by a systemd timer / entrypoint script, not a task queue.
- **No authentication/authorization layer** — the API is currently open (CORS-enabled, no API keys or OAuth).
- **No separate frontend framework** — Streamlit serves as both backend-adjacent app logic and UI.
- **No cloud-managed database** — SQLite only; PostgreSQL migration is documented as future scope, not present today.
- **No CI/CD pipeline** — tests and deployment are run manually.

12. Stack Summary (one-line version, for slides)

Python 3.11 · FastAPI + Uvicorn · Streamlit + Plotly · SQLite · pandas / NumPy / SciPy · requests (live-attempt scraping) + a calibrated synthetic pricing engine (dominant data source) · pytest · Docker + docker-compose + Nginx · deployed demo on AWS.

(Playwright and Scrapy are present in the codebase as architectural scaffolding for a future live-scraping upgrade — not as active runtime dependencies of the current build. See Section 3 for the precise

breakdown.)